How a computer sees

The plan

ACT 1

How a computer sees — we compute what a network computes, by hand.

ACT 2

How it learns — nobody programs it; it practices. With my own results.

ACT 3

What this unlocks — finding objects, cutting them out, words, generating images.

The promise: in one hour you can explain how a photo becomes the word “cat.”

A computer has never seen a cat.

A photo is just a grid of numbers

A REAL PHOTO · WHITE BOX = 10×10 PIXELS ON HER EYE

67	59	53	51	48	38	21	7	10	14
64	68	61	58	49	28	16	9	9	6
73	70	70	63	46	20	12	9	10	7
86	91	84	67	48	25	17	16	11	8
95	91	89	65	53	50	42	36	25	18
72	64	58	58	62	75	80	72	61	50
52	52	87	44	52	63	72	79	84	77
45	95	153	50	30	30	39	49	58	62
44	114	172	68	40	32	29	26	30	29
46	130	185	79	50	46	43	36	29	19

THAT BOX, ZOOMED ALL THE WAY IN · REAL VALUES

- → Each square is a **pixel**: one brightness number. 0 = black, 255 = white.
- → Color photos: three grids — red, green, blue.

This grid is ALL the computer gets. No cat, no eye — just these numbers.

LIVE ASK THE CHAT: WHICH CORNER HAS THE BIG NUMBERS — FUR OR PUPIL?

Convolution: multiply, add — done by hand

2	2	0	0	0
2	2	0	0	0
2	2	0	0	0
2	2	0	0	0
2	2	0	0	0

IMAGE · BRIGHT STRIPE (2S), DARK (0S)

×

1	0	-1
1	0	-1
1	0	-1

FILTER · A 3×3 STENCIL

=

$$2 \cdot 1 + 2 \cdot 0 + 0 \cdot (-1) = 2$$
$$2 \cdot 1 + 2 \cdot 0 + 0 \cdot (-1) = 2$$
$$2 \cdot 1 + 2 \cdot 0 + 0 \cdot (-1) = 2$$

sum = 6 loud — “edge here!”

Slide the stencil onto the flat dark area: every product is 0.
sum = 0 — silent. The filter only shouts on its pattern.

LIVE WE COMPUTE THE NINE PRODUCTS TOGETHER, OUT LOUD, BEFORE THE ANSWER

A filter is a pattern detector

INPUT PHOTO

OUR FILTER FROM LAST SLIDE · VERTICAL EDGES

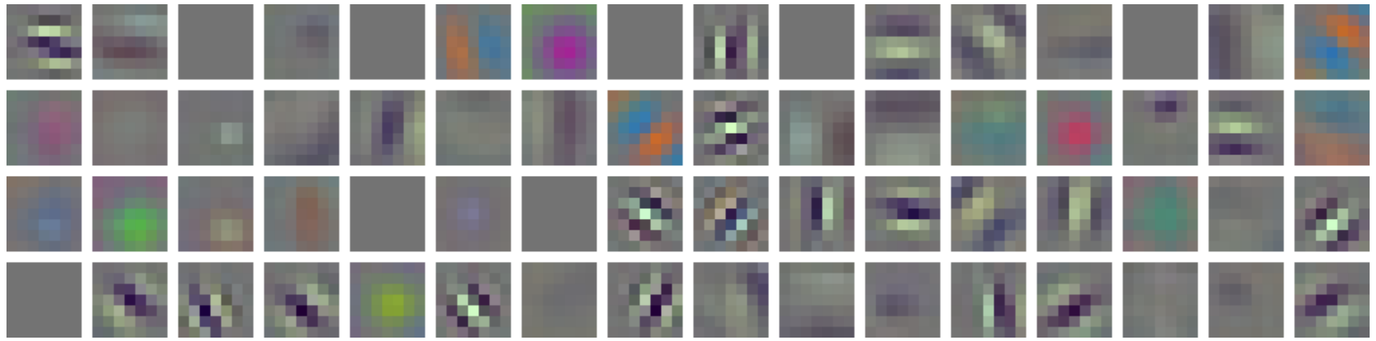
SAME FILTER, ROTATED · HORIZONTAL EDGES

White = the stencil shouted “my pattern is here.” Whiskers and eye outlines glow. Real output — this exact code ran on this exact photo.

Edges → textures → parts → objects

LAYER 1	edges & color blobs
LAYER 2	textures — fur, stripes, mesh
DEEPER	parts — ear, eye, wheel
LAST	whole objects — “cat”

Each layer looks at the previous layer’s maps, not the raw photo. This stack is a **convolutional neural network** — a CNN.



ALL 64 FIRST-LAYER FILTERS OF RESNET18 · LEARNED FROM 1.28M PHOTOS

Nobody drew these. The network discovered, by itself, that edge and color stencils are the right place to start — the same idea you just computed by hand.

The whole “eye” is one line

```
import torch.nn as nn

layer = nn.Conv2d(in_channels=3,    # red, green, blue
                  out_channels=32,  # learn 32 different stencils
                  kernel_size=3)    # each stencil is 3×3
```

One line of Python = 32 sliding stencils — exactly the operation you did by hand.

Nobody programs the filters.
The network finds them.

Learning = guess, get told how wrong, adjust

PHOTO

The hot-and-cold game: guess, get told “colder,” adjust. After enough rounds, random noise has turned into the edge detectors you saw.

Free extra photos: one cat, five lessons

ORIGINAL

MIRRORED

RANDOM CROP

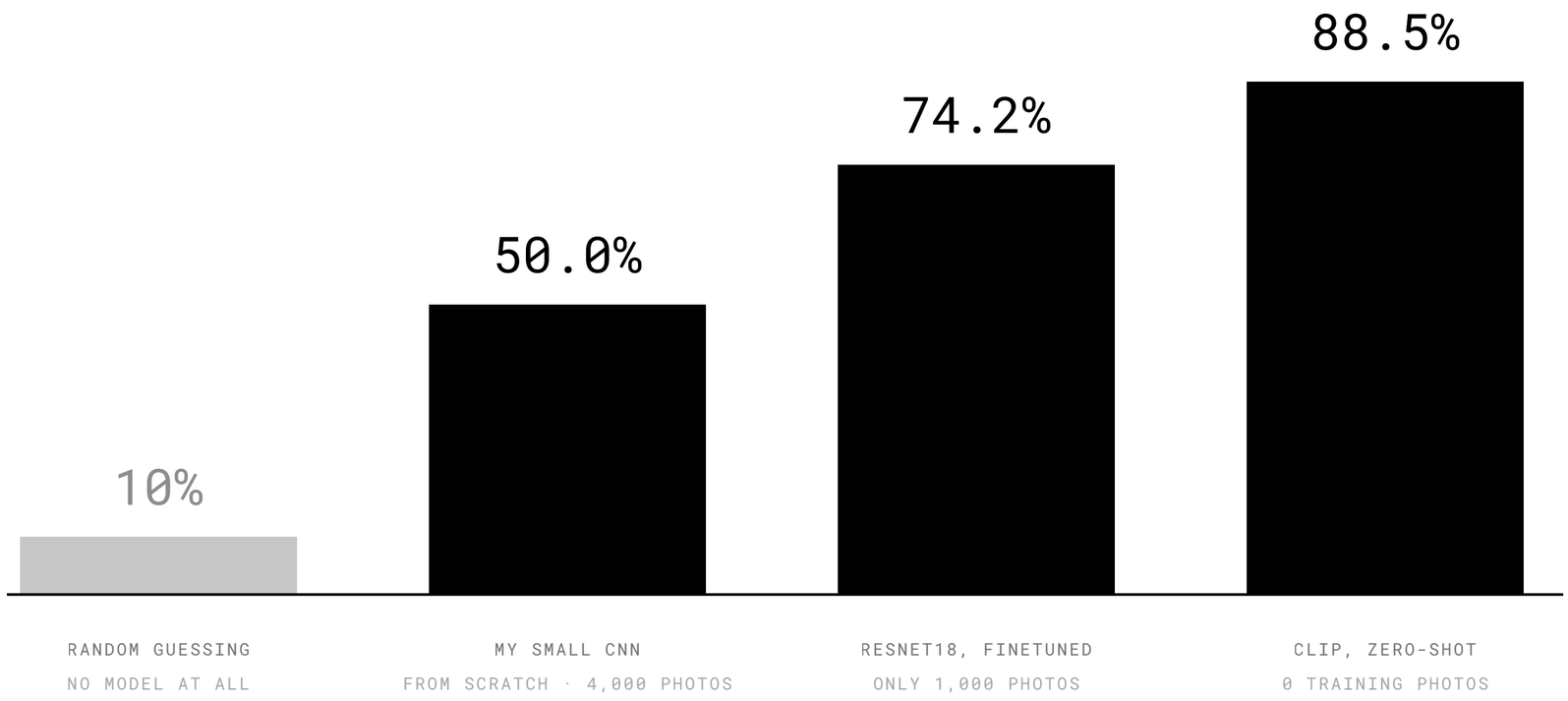
COLOR SHIFT

NOISE

```
train_tf = transforms.Compose([
    transforms.RandomCrop(32, padding=4),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
])
```

A mirrored cat is still a cat. Every epoch each photo arrives slightly different, so the network can't memorize — it must learn what cats look like. These are the exact lines from my training code.

I trained this. Here's what happened.



Same task: tiny photos, 10 categories (CIFAR-10). Accuracy on photos the models never saw.

All three are my runs, on an ordinary CPU. The notebooks rerun everything tonight if you want.

Lesson: in modern AI you stand on giants' shoulders.

LIVE BEFORE THE REVEAL: CHAT GUESSES MY FROM-SCRATCH ACCURACY · CLOSEST WINS

Borrowing a giant’s eyes

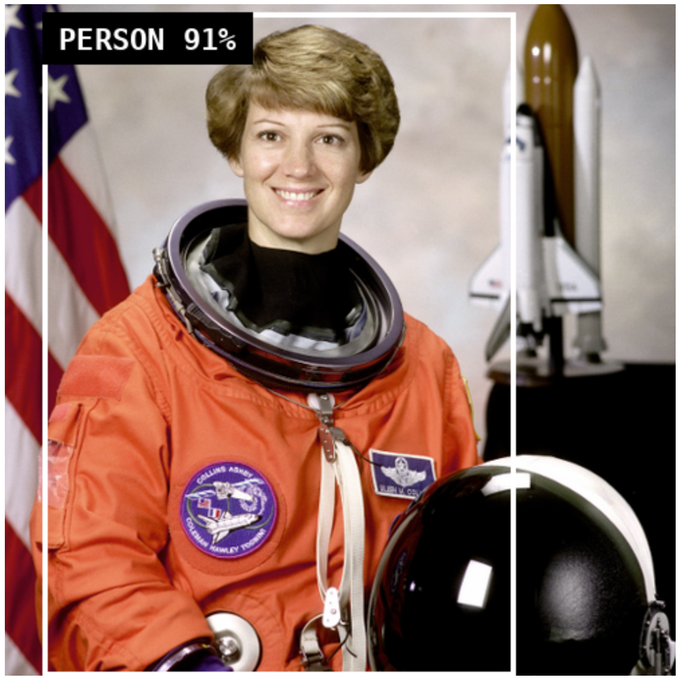
```
from torchvision.models import resnet18, ResNet18_Weights

model = resnet18(weights=ResNet18_Weights.IMAGENET1K_V1)
model.fc = nn.Linear(512, 10)  # swap the last layer: my 10 classes
```

Keep every learned filter; replace only the final layer. This is **transfer learning** — the 74.2% recipe.

Same trick.
Four superpowers.

Find it and box it

REAL OUTPUT · CAT · 97% CONFIDENT



REAL OUTPUT · PERSON · 91% CONFIDENT

- Classification says “there’s a cat somewhere.” Detection says **where** — with a box.
- One pass predicts boxes + labels + confidence — fast enough for live video.
- Self-driving cameras; your phone finding faces before it focuses.

Cut out the exact shape

PHOTO

PREDICTED MASK · WHITE = “CAT PIXEL”

MASK USED AS SCISSORS

Every single pixel gets a label: cat or not-cat. Real model output — and exactly how this video call blurs my background right now.

A model that speaks image and text

I GAVE CLIP THIS PHOTO + 4 SENTENCES

“a photo of a cat”

97.9%

“a photo of a tiger”

1.6%

“a photo of a dog”

0.5%

“a photo of a pizza”

0.0%

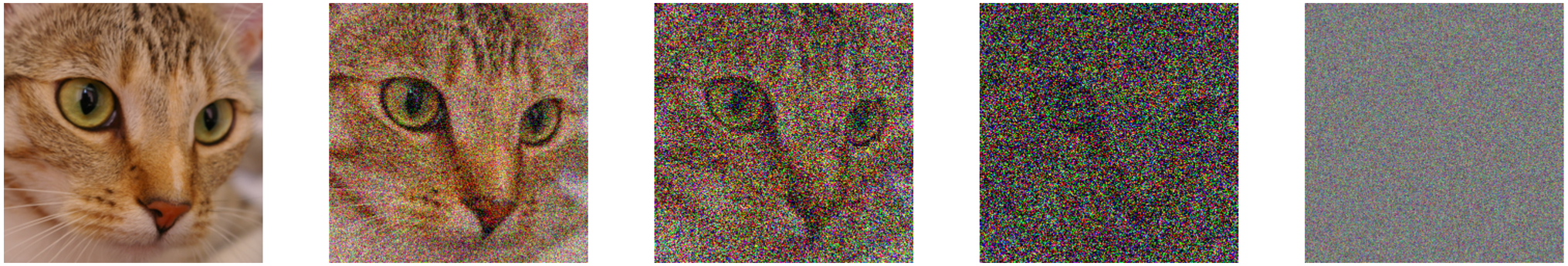
CLIP learned from **400 million** internet photos with their captions. It scores how well a sentence matches an image — so you classify with **sentences**, no training. That’s “zero-shot.”

```
inputs = processor(
    text=["a cat","a dog","a pizza"],
    images=photo, return_tensors="pt")
probs = model(**inputs) \
    .logits_per_image.softmax(dim=1)
```

CODE CARD 4 OF 4 · THE 88.5% MODEL

Start from static

TRAINING: RUIN THE PHOTO, STEP BY STEP → (REAL NOISE, REALLY ADDED)



← GENERATION: START FROM PURE STATIC, LEARN TO UNDO ONE STEP AT A TIME

The network practices one humble skill — remove a little noise. Apply it over and over to fresh static and an image crystallizes. That's **diffusion**, the engine of AI image generators.

Not magic. A syllabus.

- → Everything today is on the official **IOAI 2026 syllabus** — the high-school AI olympiad I'm training for.
- → My two notebooks rerun it all: the CNN, the 50.0 → 74.2 → 88.5 ladder. They're yours.
- → You multiplied small numbers and ran a convolution. So yes — you can do this.

Questions — then I'll open the notebooks live.